

Problem Session 08: Lock Mechanisms

Peterson's Algorithm · LL/SC · MCS Lock

CPE 333 Operating Systems, 1/2026 — Group OS InW

Group: OS InW

67070501075

Siriwan Yindeephon

How We Evaluate
Conclusion

67070501018

Tithinan Sobking

Peterson's Algorithm

67070501005

Kiatisak Markmeeshap

Load-Linked / Store-Conditional

67070501021

Thanaboon Tikaew

MCS Lock

67070501040

Worawut Sereethai

Comparison: Throughput

67070501059

Chanon Lhumsa-ard

Comparison: Summary

How We Evaluate

67070501075

Siriwan Yindeephoto

Metrics from Lecture 8

Metric (Lecture 8)	Question	What we measure
Mutual exclusion	Is only one thread inside at a time?	Shared counter must equal total acquisitions: lost = total – counter, expected 0
Absence of deadlock	Does some waiting thread always get in?	Every run must finish when the time is up (a hang would show)
Fairness	Does every thread get a fair chance?	Fewest ÷ most acquisitions per thread (1.0 = equal, near 0 = severe imbalance)
Performance	How much time does the lock cost?	Acquisitions per second over the measurement window, 1 to 8 threads

Argument vs. Experiment

Property	Argument (why it should hold)	Experiment (what we observed)
Mutual exclusion	An invariant: at most one thread is past lock_acquire	No violation observed in 390 runs: lost = 0
Absence of deadlock	Some waiting thread can always proceed	No violation observed in 390 runs: every run finished
Fairness	Bounded waiting or FIFO order, if the algorithm has it	Acquisition counts per thread in a 1 s window; the scheduler affects them

- The arguments assume: sequentially consistent memory (or the right fences), a scheduler that eventually runs every thread, and a holder that calls unlock
- The no-fence Peterson demo is the one build that is expected to fail

The Benchmark Loop

- All threads start together at a barrier
- Loop until main raises stop after 1 s
- counter is a plain variable: two threads inside together lose an update
- Each thread counts its own acquisitions (n) and timestamps its start and stop
- Window = first thread's start to last thread's stop; creating and joining threads is outside it
- Same loop for every lock: only lock_acquire / lock_release change

```
static void *worker(void *arg)
{
    int id = (int)(long)arg;
    long n = 0;

    pthread_barrier_wait(&start);
    t_begin[id] = now();                /* counting starts here */
    while (!stop) {
        lock_acquire(id);
        counter = counter + 1;          /* critical section */
        lock_release(id);
        n++;
    }
    t_end[id] = now();                  /* ... and stops here */
    acquired[id] = n;
    return NULL;
}
```

Reading One Run

```
--- tas threads=4 all-cpus run 1 ---  
$ bin/tas 4 1  
lock=tas threads=4 cpus=8  
  thread 0: 1049655 acquisitions  
  thread 1: 4181859 acquisitions  
  thread 2: 2232569 acquisitions  
  thread 3: 856272 acquisitions  
total=8320355 counter=8320355 lost=0  
window=1.000 s throughput=8318015 acq/s fairness(min/max)=0.205
```

- Mutual exclusion: total 8,320,355 = counter
8,320,355 → lost = 0
- Absence of deadlock: the run finished
- Fairness: $856,272 \div 4,181,859 = 0.205$
- Performance: 8.3 M acquisitions per second

Experiment Setup

Setting	Values
Locks	Peterson, LL/SC, MCS
References (Lecture 8)	TAS spin lock, pthread_mutex
Threads	1, 2, 4, 8 (Peterson: 1, 2)
CPUs	all 8, and one (taskset -c 0)
Runs	10 × 1 s per setting, median shown (spread: Run-to-Run slide)

- WSL2 Ubuntu, kernel 6.18.33.2-microsoft-standard-WSL2
- 8 CPUs
- gcc 15.2.0 -O2
- LL/SC on AArch64: qemu-aarch64 10.2.1
- WSL2 vCPUs are scheduled by Windows too, so absolute numbers will differ on other machines
- Tiny critical section, no work outside it, 1 s runs, 10 repetitions: compare locks with each other, not with other machines
- One CPU shows what spinning costs when the lock holder cannot run

Peterson's Algorithm

67070501018

Tithinan Sobking

Peterson's Algorithm: Idea

- Pure software lock for two threads: only loads and stores
- $\text{flag}[i] = 1$ — “thread i wants to enter”
- $\text{turn} = \text{other}$ — “you go first”
- Wait while the other wants in and it is the other's turn
- The thread that wrote turn last waits
- Reference build: `_Atomic ints, seq_cst`; `FENCE()` is empty there (the `seq_cst` stores order the loads)

```
typedef _Atomic int shared_t;
static shared_t flag[2];      /* flag[i] = 1: thread i wants in */
static shared_t turn;         /* whose turn it is to wait */

void lock_acquire(int self)
{
    int other = 1 - self;
    flag[self] = 1;           /* I want to enter ... */
    turn = other;             /* ... but you go first */
    FENCE();                  /* stores done before the loads */
    while (flag[other] == 1 && turn == other)
        ;                    /* spin: you want in, your turn */
}

void lock_release(int self)
{
    flag[self] = 0;
}
```

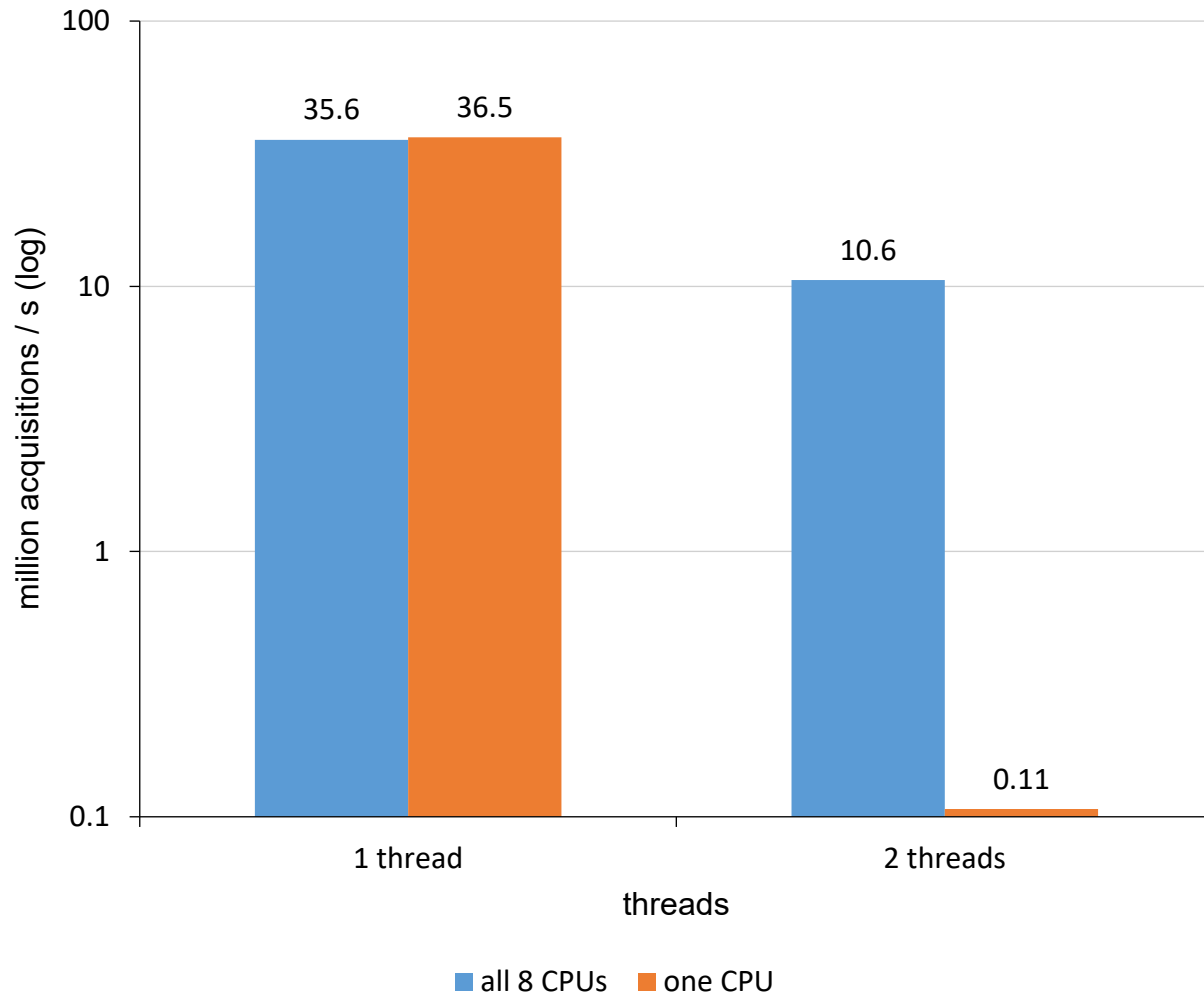
Peterson: Mutual Exclusion and Deadlock

- Mutual exclusion (assuming sequential consistency): if both were inside, take the thread P that wrote turn last; the other thread's `flag[] = 1` came before its own turn write, so P reads `flag[other] == 1` and `turn == other`, and waits — contradiction
- No deadlock: both waiting would need `turn == 1` and `turn == 0` at once; and the holder must call unlock
- x86 lets a load pass an earlier store (store buffer): both can read `flag[other] == 0` and both enter. That breaks the assumption, not the logic

Build (2 threads)	CPUs	Runs with lost updates	Lost updates per run
volatile, no fence	all 8	10 / 10	17,178 – 237,149
volatile, no fence	one	0 / 10	0
volatile + FENCE()	all 8	0 / 10	0
_Atomic, seq_cst (reference)	all 8	0 / 10	0

- FENCE() = `lock orq $0x0, (%rsp)`, a full barrier. The volatile builds are data races in C11: they show GCC on x86 only

Peterson: Performance



- 1 thread: 36 M acq/s — no contention
- 2 threads, all CPUs: 11 M acq/s — flag and turn move between cores
- 2 threads, one CPU: 107 k acq/s — the waiter spins until the scheduler switches
- Busy-waiting costs a whole time slice when the other thread is not running

Peterson: Fairness and Limitations

- All CPUs, 2 threads: fairness 1.00 — turn gives way to the other thread, so a waiter enters after at most one entry by the other (bounded waiting)
- One CPU, 2 threads: fairness 0.001 (run 1: 126 vs 103,221) — the imbalance comes from the scheduler's time slices; bounded waiting is a property of the algorithm, not of this window
- Limitations:
 - Two threads only (N threads need a tournament of Peterson locks or the filter lock)
 - Needs sequential consistency: a full fence on x86 — the textbook volatile version is broken there
 - Spins instead of sleeping

Load-Linked / Store-Conditional

67070501005

Kiatisak Markmeeshap

LL/SC: Idea

- LoadLinked(p): load *p and watch the address
- StoreConditional(p, v): store only if nobody stored to p since the LoadLinked
- Lock: spin on LL until 0, then SC 1; if SC fails, start over
- MIPS, Alpha, PowerPC, ARM (AArch64: LDXR/STXR)
- x86 has no LL/SC — it offers CAS (lock cmpxchg)

```
static inline int LoadLinked(volatile int *p)
{
    int v;
    __asm__ volatile("ldaxr %w0, [%1]"
                     : "=r"(v) : "r"(p) : "memory");
    return v;
}

static inline int StoreConditional(volatile int *p, int v)
{
    int fail;
    __asm__ volatile("stxr %w0, %w2, [%1]"
                     : "=r"(fail) : "r"(p), "r"(v) : "memory");
    return !fail;
}

void lock_acquire(int id)
{
    (void)id;
    while (1) {
        while (LoadLinked(&flag))
            ; /* spin until it is zero */
        if (StoreConditional(&flag, 1))
            return; /* no store in between: ours */
        /* else another thread won: try again */
    }
}
```

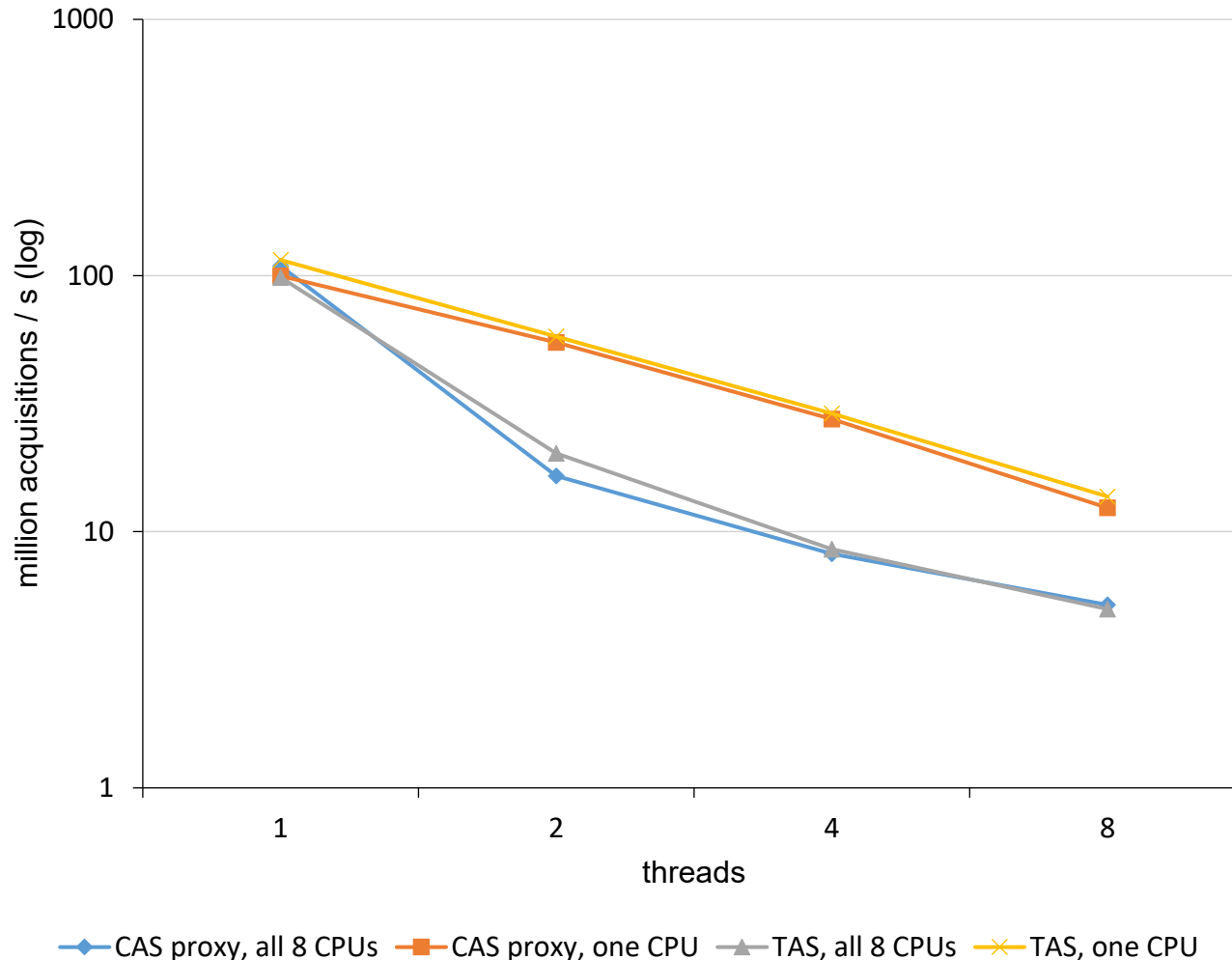
LL/SC: Mutual Exclusion and Deadlock

- Mutual exclusion: two threads may both LL a 0, but the first SC writes the flag, so the second SC fails
- No deadlock: an SC fails only if another store got in (that thread now holds the lock) or spuriously (e.g. an interrupt) — the loop retries
- AArch64 build under qemu-aarch64 10.2.1, 4 threads: no violation observed in 10 / 10 runs (lost = 0, 5,107,705+ acquisitions each)

```
0000000000400bcc <lock_acquire>:  
adrp x1, 4b1000  
add x1, x1, #0xf40  
mov w2, #0x1  
nop  
nop  
ldaxr w0, [x1]  
cbnz w0, 400be0 <lock_acquire+0x14>  
stxr w0, w2, [x1]  
cbnz w0, 400be0 <lock_acquire+0x14>  
ret
```

ldaxr = Load-Linked, stxr = Store-Conditional (writes 0 to w0 on success)

LL/SC: Performance



- Timed on x86: a CAS spin lock, NOT hardware LL/SC (qemu speed is meaningless)
- All CPUs: 109 M acq/s with 1 thread → 5.2 M with 8 — every try pulls the flag's cache line
- One CPU, 8 threads: 12 M acq/s
- Waiters only read until the flag looks free, then try once (the LL/SC pair on Arm)

LL/SC: Fairness and Limitations

- No queue: the first SC to land wins — fairness at 8 threads 0.55 (all CPUs), 0.00 (one CPU; run 1: the unluckiest thread got 1 of 13,150,964)
- A thread preempted while holding the lock makes every other thread spin out its time slice
- Limitations:
 - Starvation is possible; spins instead of sleeping
 - Hardware-specific: not on x86; SC may fail spuriously
 - Little may run between LL and SC, or the reservation is lost
- Advantage over CAS: SC fails on any store, even one that writes back the same value (no ABA problem)

MCS Lock

67070501021

Thanaboon Tikaew

MCS Lock: Idea

- Mellor-Crummey and Scott, 1991; Dijkstra Prize 2006
- Waiters form a queue: a linked list of nodes, tail points to the last
- Join: one atomic exchange on tail, then link in behind the previous node
- Each thread spins on its OWN node; the one ahead clears it on release
- Linux's spinlock (qspinlock, since v4.2) is built on MCS

```
struct node {
    struct node *next;           /* the thread queued behind me */
    int locked;                  /* 1 = keep waiting */
} __attribute__((aligned(64))); /* one cache line per node */

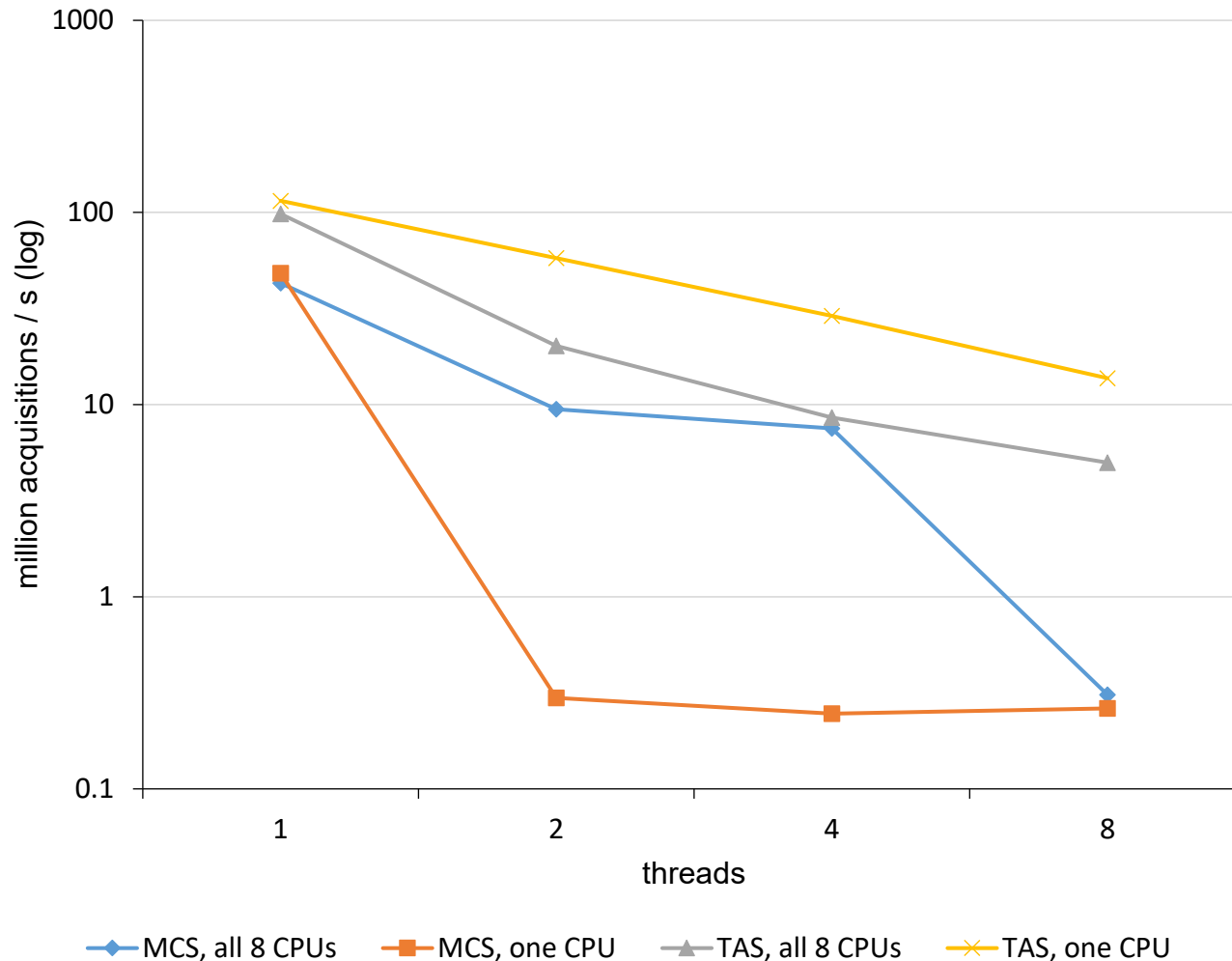
void lock_acquire(int id)
{
    struct node *me = &qnode[id];
    me->next = NULL;
    me->locked = 1;
    struct node *prev = __atomic_exchange_n(&tail, me, __ATOMIC_ACQ_REL);
    if (prev != NULL) {          /* queue not empty: */
        STORE(&prev->next, me); /* link in behind prev */
        while (LOAD(&me->locked))
            ;                    /* spin on MY node only */
    }
}
```

MCS: Mutual Exclusion and Deadlock

- Mutual exclusion: the exchange gives every thread one place in one queue; only the head is inside, and it wakes exactly one node
- No deadlock: the holder always hands over — to the next node, or resets tail when nobody waits
- release() waits for a waiter that has swapped tail but not linked in yet
- No violation observed in 80 runs (1–8 threads, all CPUs and one CPU): lost = 0, every run finished

```
void lock_release(int id)
{
    struct node *me = &qnode[id];
    struct node *next = LOAD(&me->next);
    if (next == NULL) {
        struct node *expect = me;
        if (__atomic_compare_exchange_n(&tail, &expect, NULL, 0,
                                         __ATOMIC_RELEASE, __ATOMIC_RELAXED))
            return; /* nobody waiting: lock is free */
        while ((next = LOAD(&me->next)) == NULL)
            ; /* a waiter is still linking in */
    }
    STORE(&next->locked, 0); /* hand the lock to the next thread */
}
```

MCS: Performance



- Each waiter spins on its own cache line; a handover touches one line
- All CPUs, 8 threads: 309 k acq/s (TAS: 5.0 M)
- One CPU, 8 threads: 262 k acq/s — the next in the queue is not running
- In the kernel, spinlocks run with preemption disabled, so a queued waiter is never descheduled

MCS: Fairness and Limitations

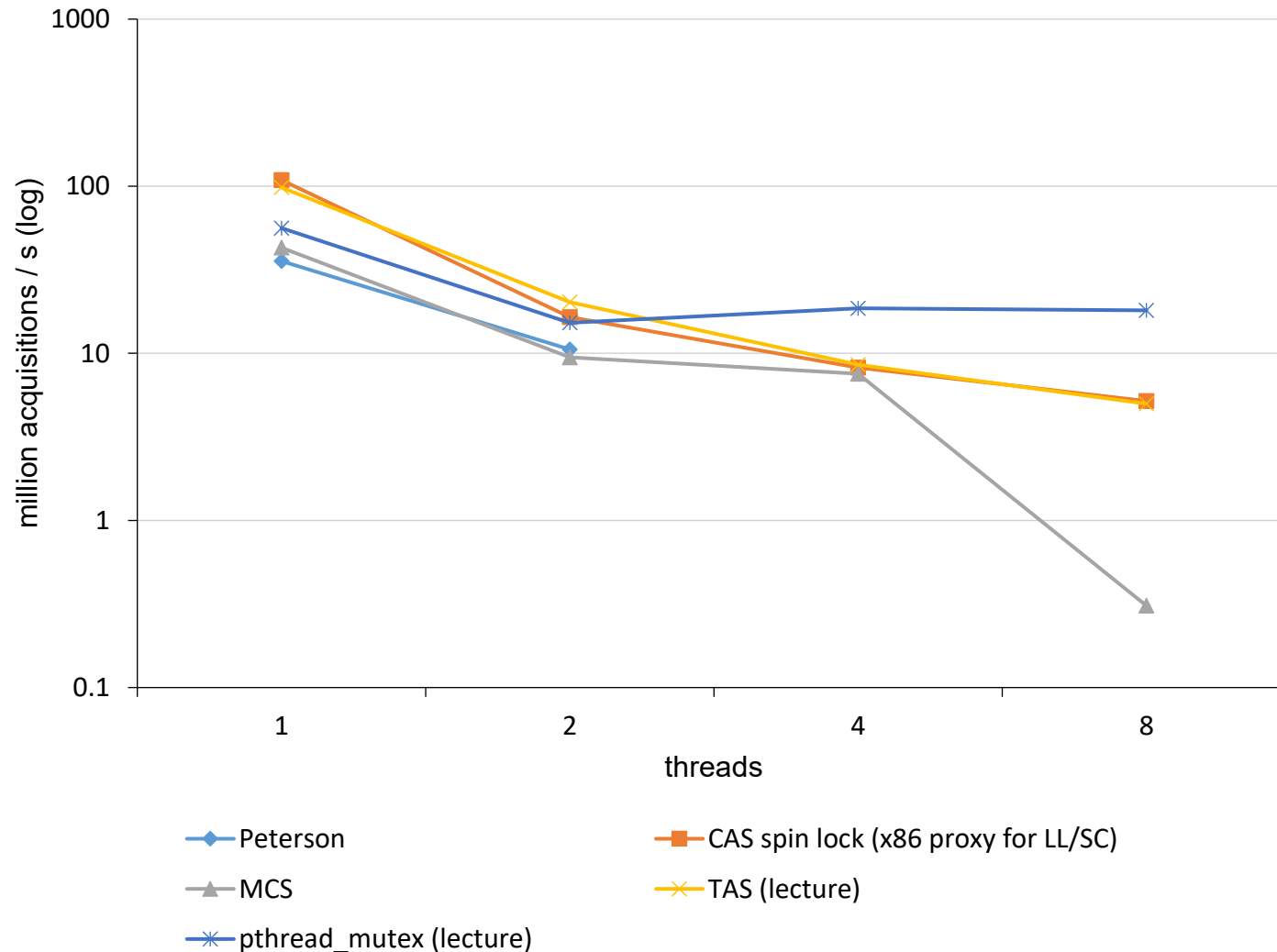
- First-come, first-served through the queue: fairness 1.00 / 1.00 / 0.96 at 2/4/8 threads (all CPUs) — the fairest lock at 8 threads
- Limitations:
 - Needs an atomic exchange (and CAS for release) from the hardware
 - One queue node per waiting thread, passed in or kept per thread
 - A preempted waiter stalls everyone behind it — the kernel disables preemption
 - On one CPU here: 262 k acq/s at 8 threads, fairness 0.00 (run 1: 10 vs 146,478) — the scheduler, not the queue order

Comparison: Throughput

67070501040

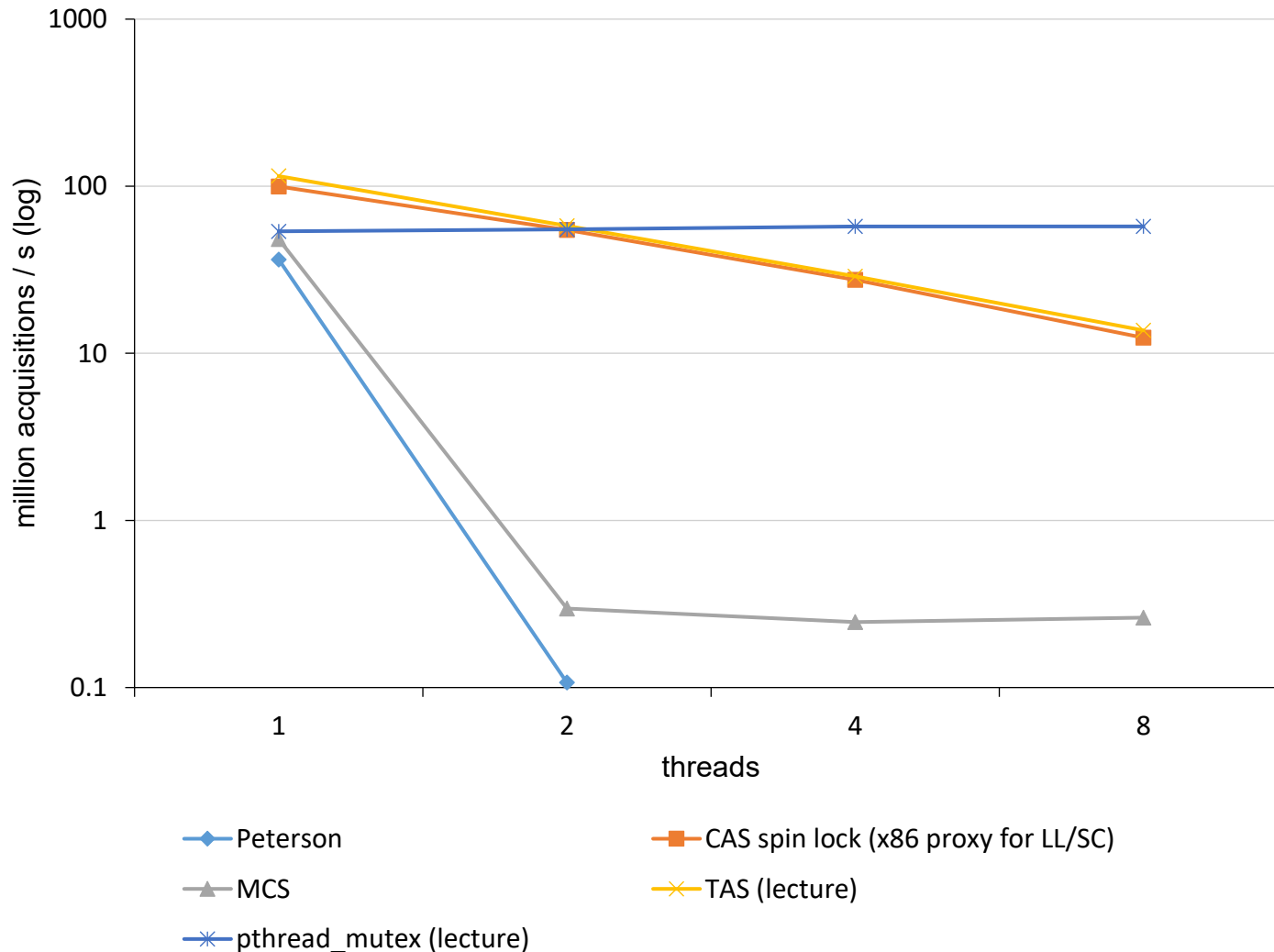
Worawut Sereethai

Throughput on All 8 CPUs



- Medians of 10 runs. 1 thread, no contention: CAS spin lock 109 M and TAS 98 M acq/s are fastest
- More threads: every spin lock slows down as threads fight over cache lines
- 8 threads: MCS 309 k, CAS spin 5.2 M, TAS 5.0 M acq/s
- Fastest at 8 threads: pthread_mutex, 18 M acq/s

Throughput on One CPU



- TAS / CAS spin stay fast (14 M / 12 M at 8 threads), but one thread holds on while the others get almost nothing
- Peterson and MCS hand the lock to a thread that is not running: 107 k (Peterson, 2 thr), 262 k (MCS, 8 thr)
- pthread_mutex sleeps instead of spinning: 54 M → 57 M acq/s from 1 to 8 threads

Comparison: Summary

67070501059

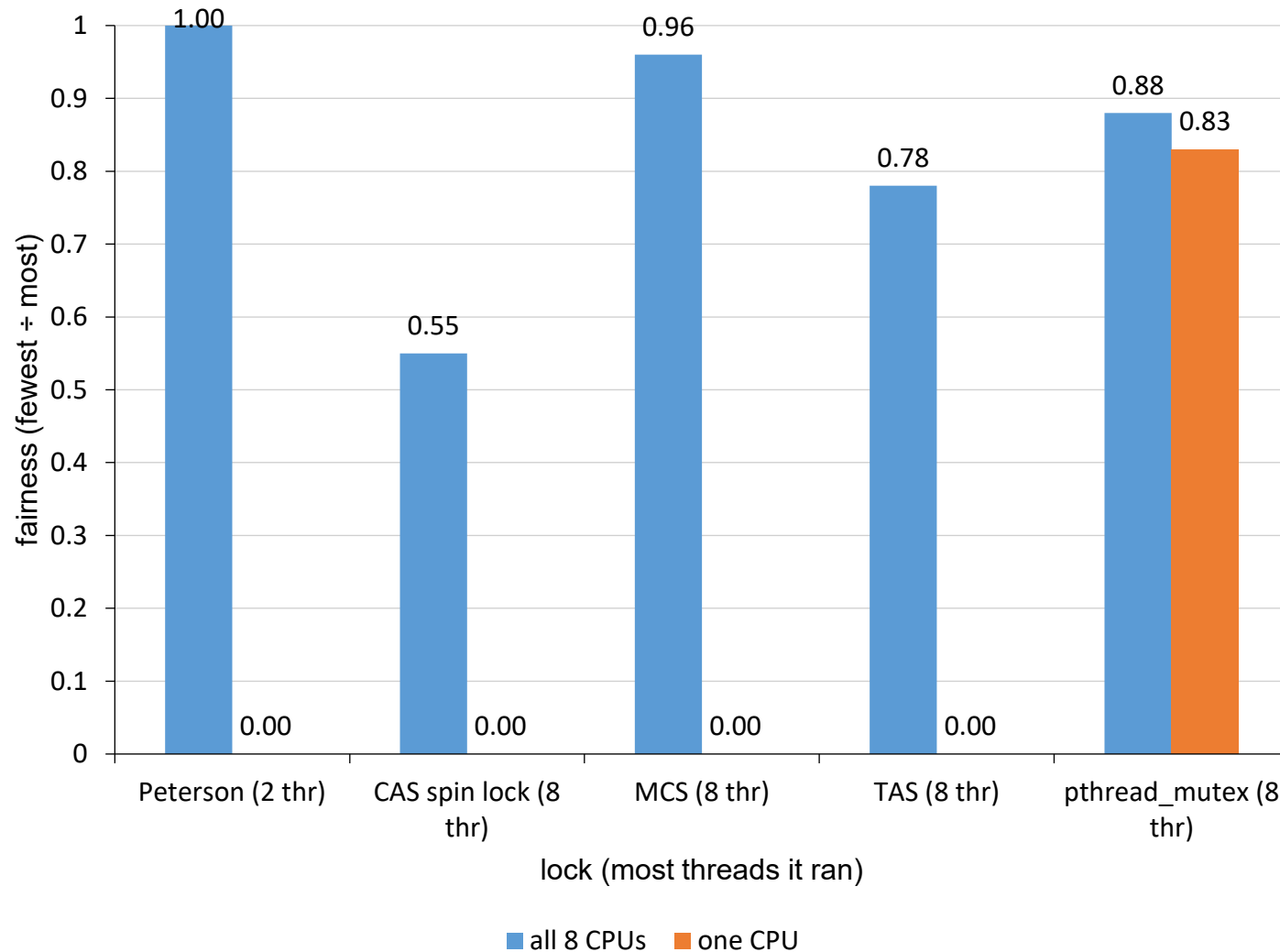
Chanon Lhumsa-ard

Correctness: Mutual Exclusion and Deadlock

Lock	Runs	Lost updates	Runs finished	Also tested
Peterson	60	0	60 / 60	volatile, no fence: lost updates in 10 / 10 runs
CAS spin lock	80	0	80 / 80	x86 = CAS proxy; real ldaxr/stxr under qemu: lost = 0 in 10 / 10 runs
MCS	80	0	80 / 80	
TAS	80	0	80 / 80	
pthread_mutex	80	0	80 / 80	

- Mutual exclusion: lost = total acquisitions – counter, summed over all runs of the correct builds
- Peterson held on x86 only with the fence (or `_Atomic seq_cst`)

Fairness



- All CPUs: MCS 0.96 is the fairest at 8 threads — its queue is first-come, first-served
- TAS 0.78, CAS spin 0.55: whoever wins the race gets in
- One CPU: severe imbalance for every spin lock — the scheduler decides who runs, e.g. MCS 10 vs 146,478, TAS 1 vs 6,834,831 acquisitions (run 1, fewest vs most)
- Only pthread_mutex stays balanced on one CPU: 0.83
- Thread-runs with zero acquisitions: 0 of 1260

Run-to-Run Variation

Lock (threads)	CPUs	slowest run	median	fastest run	max ÷ min
Peterson (2)	all 8	10 M	11 M	14 M	1.4×
CAS spin lock (8)	all 8	4.2 M	5.2 M	5.3 M	1.3×
MCS (8)	all 8	155 k	309 k	1.7 M	11.0×
TAS (8)	all 8	4.8 M	5.0 M	5.9 M	1.2×
pthread_mutex (8)	all 8	18 M	18 M	18 M	1.0×
Peterson (2)	one	58 k	107 k	564 k	9.7×
CAS spin lock (8)	one	11 M	12 M	13 M	1.1×
MCS (8)	one	59 k	262 k	509 k	8.6×
TAS (8)	one	13 M	14 M	15 M	1.1×
pthread_mutex (8)	one	54 M	57 M	59 M	1.1×

- acq/s per run; 10 runs per setting, no confidence intervals

Summary

Lock	Needs	Mutual excl.	Deadlock-free	Fairness	acq/s, all 8 CPUs	acq/s, one CPU
Peterson (2 thr)	loads/stores + seq_cst	yes	yes	1.00	11 M	107 k
CAS spin lock (8 thr)	CAS (LL/SC on Arm)	yes	yes	0.55	5.2 M	12 M
MCS (8 thr)	atomic exchange + CAS	yes	yes	0.96	309 k	262 k
TAS (8 thr)	test-and-set	yes	yes	0.78	5.0 M	14 M
pthread_mutex (8 thr)	atomics + futex	yes	yes	0.88	18 M	57 M

- Fairness = fewest ÷ most acquisitions per thread; medians of 10 runs of 1 s; mutual exclusion and deadlock: no violation observed
- CAS spin lock = x86 stand-in for LL/SC: its speed is not the speed of LL/SC hardware
- Peterson held on x86 only with the fence (or `_Atomic seq_cst`)

Conclusion

67070501075

Siriwan Yindeephot

Conclusion

- Arguments plus no violation observed in 390 runs — Peterson only with sequential consistency (a fence on x86)
- Peterson: software only, bounded waiting, but two threads only
- LL/SC: one short loop; our x86 numbers are a CAS proxy; no queue, so unbalanced
- MCS: a queue gives first-come, first-served for any number of threads; Linux builds on it
- Spinning on one CPU wastes time slices; sleeping locks (futex, pthread_mutex) avoid it
- Limits: WSL2 vCPUs, tiny critical section, 10×1 s runs; some results vary widely

References

- R. H. Arpaci-Dusseau, A. C. Arpaci-Dusseau, Operating Systems: Three Easy Pieces, ch. 28 “Locks”
- CPE 333 Lecture 8: Locks Mechanism
- G. L. Peterson, “Myths About the Mutual Exclusion Problem”, Information Processing Letters 12(3), 1981
- J. M. Mellor-Crummey, M. L. Scott, “Algorithms for Scalable Synchronization on Shared-Memory Multiprocessors”, ACM TOCS 9(1), 1991
- Linux kernel source: `kernel/locking/qspinlock.c`
- Arm Architecture Reference Manual for A-profile architecture: LDAXR, STXR
- Code and raw results: PS08/src, PS08/results